

Claude Code — Bootstrapping Guide for New Users

A practical guide to using AI coding agents effectively, built from real-world workflow experience.

Table of Contents

1. [What is Claude Code?](#)
 2. [Windows Users: WSL Setup](#)
 3. [Setup & Prerequisites](#)
 4. [Understanding AI Models](#)
 5. [Using This Repo to Bootstrap Projects](#)
 6. [Your First Project](#)
 7. [The Core Workflow](#)
 8. [Working with CLAUDE.md](#)
 9. [The Multi-AI Review Process](#)
 10. [Git & GitHub Practices](#)
 11. [Scaling to Bigger Projects](#)
 12. [Documentation as a Superpower](#)
 13. [Dealing with AI Limitations](#)
 14. [Prompt Patterns That Work](#)
 15. [Common Mistakes to Avoid](#)
-

What is Claude Code?

Claude Code is a command-line AI coding agent made by Anthropic. You run it in your terminal, describe what you want to build, and it writes code, creates files, runs commands, and manages your project — all through conversation.

It's not just autocomplete. It's an agent that can: - Scaffold entire projects from a description - Read and understand your existing code - Edit multiple files to implement features - Run tests, fix bugs, and refactor - Manage git commits and create PRs - Enter "plan mode" to think before coding

You stay in control — you describe what you want, review what it does, and guide direction.

Windows Users: WSL Setup

Claude Code requires a Unix-like environment. On Windows, you'll use **WSL (Windows Subsystem for Linux)** — it gives you a full Linux terminal inside Windows.

Installing WSL

```
# Open PowerShell as Administrator and run:  
wsl --install
```

This installs Ubuntu by default. **Restart your PC** when prompted.

After restart, open "Ubuntu" from the Start menu. It will ask you to create a Linux username and password (these are separate from your Windows login).

Important WSL Rules

1. **Always work in the Linux filesystem.** Store your projects in `~/projects/`, not in `/mnt/c/Users/...`. The Linux filesystem is much faster for development.
2. **Use the WSL terminal for everything.** Don't mix PowerShell and WSL commands. Once you open Ubuntu, stay there.
3. **VS Code integration** (recommended): Install VS Code on Windows, then install the "Remote - WSL" extension. Now you can type `code .` from any WSL directory and VS Code opens connected to your Linux environment.
4. **Terminal copy/paste:** Use `Ctrl+Shift+C` and `Ctrl+Shift+V` inside the WSL terminal (not `Ctrl+C` / `Ctrl+V`).

WSL Package Manager

WSL/Ubuntu uses `apt` instead of `brew`:

```
# Update package lists (do this first)
sudo apt update && sudo apt upgrade -y

# Install essentials
sudo apt install -y build-essential curl wget
```

Accessing Files Between Windows and WSL

```
# From WSL, Windows files are at:
ls /mnt/c/Users/YourWindowsUsername/

# From Windows Explorer, WSL files are at:
# \\wsl$\Ubuntu\home\your-linux-username\
# (type this in the Explorer address bar)
```

Common WSL Issues

- **npm permission errors:** Run `sudo chown -R $(whoami) ~/.npm`
- **Slow file operations:** You're probably working in `/mnt/c/` — move to `~/`
- **Git line endings:** Run `git config --global core.autocrlf input` to avoid Windows line ending issues

Setup & Prerequisites

Required Tools

Install these before your first session. Instructions are for **WSL/Ubuntu** (Windows users). macOS users can substitute `apt` commands with `brew install` equivalents.

1. System essentials

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y build-essential curl wget git
```

2. Node.js (v18 or later) — needed for Codex and Gemini CLIs

```
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
sudo apt-get install -y nodejs

# Verify:
node --version
```

3. Git configuration

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
git config --global core.autocrlf input # Prevents Windows line-ending issues
```

4. GitHub CLI

```
curl -fsSL https://cli.github.com/packages/githubcli-archive-keyring.gpg \
| sudo dd of=/usr/share/keyrings/githubcli-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) \
signed-by=/usr/share/keyrings/githubcli-archive-keyring.gpg] \
https://cli.github.com/packages stable main" \
| sudo tee /etc/apt/sources.list.d/github-cli.list > /dev/null
sudo apt update && sudo apt install -y gh

# Authenticate:
gh auth login
# Follow the prompts
```

5. Claude Code — native installer (recommended, auto-updates)

```
curl -fsSL https://claude.ai/install.sh | bash
# You'll need an Anthropic API key or Claude Max subscription
```

6. Codex CLI (for code review)

```
npm install -g @openai/codex
# You'll need an OpenAI API key or ChatGPT Plus/Pro/Team subscription
```

7. Gemini CLI (for code review)

```
npm install -g @google/gemini-cli
# You'll need a Google account (free tier available)
```

Verify Everything Works

```
git --version
node --version
gh --version
claude --version
codex --version
gemini --help
```

Understanding AI Models

This is important: **AI models have a knowledge cutoff date.**

Tool	Underlying Model(s)	What This Means
Claude Code	Claude Opus 4.6 / Sonnet 4.6	Anthropic's models — strong at planning, coding, reasoning
Codex CLI	OpenAI GPT/o-series models	OpenAI's models — different strengths, good for second opinions
Gemini CLI	Google Gemini models	Google's models — another perspective for reviews

Why the cutoff matters:

- **Dependencies:** The AI might suggest an older version of a library. Always say “use the latest stable version” in your prompts.
- **New APIs:** If a framework released new features after the cutoff, the AI won't know about them unless you tell it or provide docs.
- **Breaking changes:** Libraries evolve. If something doesn't work, it might be suggesting deprecated syntax.

How to work around it:

- Explicitly state version requirements: *“Use React 19 with the latest stable Next.js”*
- Use Context7 (Claude can fetch live documentation during your session)
- When in doubt, check official docs yourself and paste relevant info into the chat

Using This Repo to Bootstrap Projects

This repo isn't just documentation — it's a practical bootstrapping tool. You can point Claude Code at this guide whenever you start a new project, so it follows the workflow from day one.

One-Time Setup: Clone This Repo

```
# Clone to your home directory so it's always available
cd ~
git clone https://github.com/daveremy/claude-code-guide.git
```

Starting a New Project

```
# 1. Create your new project directory
mkdir ~/projects/my-new-app
cd ~/projects/my-new-app

# 2. Start Claude Code
claude --dangerously-skip-permissions
```

Then paste this as your **first prompt** (customize the bracketed parts):

Read the full guide and cheat sheet in this repo and internalize the workflow, best practices, and conventions described:
<https://github.com/daveremy/claude-code-guide>

This is my development workflow – follow it for this entire project.

Key points:

- Always plan before coding (plan mode)
- I will review plans with Codex and Gemini before you implement
- After implementation, I will run /simplify and do multi-AI code reviews
- Use git with frequent commits
- Maintain CLAUDE.md, README.md, and ROADMAP.md (for bigger projects)
- Use latest stable versions of all dependencies
- Documentation is critical – keep it updated

Now, here is what I want to build: [YOUR PROJECT DESCRIPTION]

Features:

- [feature 1]
- [feature 2]
- [feature 3]

Tech stack: [your preferences, or "suggest appropriate tech"]

Please enter plan mode and create an implementation plan.

Why This Works

When Claude reads the guide, it internalizes: - The full plan → review → build → simplify → review → test → commit workflow - The convention of using CLAUDE.md, README.md, and ROADMAP.md - Best practices for dependency versions and documentation - The phased development approach for larger projects - The multi-AI review process and when it happens

The key points in the prompt reinforce the most important parts, so Claude doesn't just skim — it understands what you expect at every stage.

This means you don't have to remember or re-explain the workflow every time — it's baked in from the first prompt.

Alternative: Local File Reference

If you cloned the repo locally, you can reference files directly (faster, no network fetch):

Read ~/claude-code-guide/claude-code-guide.md and
~/claude-code-guide/claude-code-cheatsheet.md – internalize
the workflow and follow it for this project.

Now, here is what I want to build: [description] ...

The Core Workflow

This is the process to follow for every feature or project. It may seem like a lot of steps, but each one prevents problems and saves time.

Step 1: Describe & Plan

Write a clear description of what you want to build. Be specific. Then ask Claude to enter plan mode.

Example prompt:

```
I want to build a personal expense tracker web app.
```

```
Features:
```

- Add expenses with amount, category, and date
- View expenses in a table with sorting
- Filter by category and date range
- Show monthly spending chart
- Data stored in local storage (no backend for now)

```
Tech: React with Vite, Tailwind CSS for styling, Recharts for charts.
```

```
Please enter plan mode and create an implementation plan.
```

Claude will analyze your request and produce a structured plan — what files to create, what components to build, what order to implement things.

Why plan mode matters: Without it, Claude might start coding immediately and make architectural decisions you disagree with. Planning first means you catch issues before any code is written.

Step 2: Review the Plan with Multiple AIs

This is a key practice: **don't rely on a single AI's opinion.**

Take Claude's plan and ask Codex and Gemini to review it:

```
Review this implementation plan for a personal expense tracker.  
Look for: missing edge cases, better architectural choices,  
potential issues, and anything you'd do differently.
```

```
[paste the plan]
```

Each AI has different training and different blind spots. Cross-referencing catches things a single model would miss.

Loop until satisfied: If a reviewer suggests meaningful improvements, bring those back to Claude, revise the plan, and review again. Usually 1-2 rounds is enough.

Step 3: Implement

Once the plan is solid, tell Claude to proceed with implementation. Claude will: - Create the project structure - Write the code - Install dependencies - Set up configuration

Let it work, but pay attention to the output. If something looks off, speak up immediately.

Step 4: Simplify

After implementation, run the code simplifier:

```
/simplify
```

This reviews the code Claude just wrote and looks for: - Unnecessary complexity - Code that could be cleaner - Duplicated logic - Opportunities for better patterns

Loop this until the simplifier reports no more substantial improvements. Usually 2-3 rounds.

Step 5: Code Review (Again with Multiple AIs)

Now review the actual code with Codex and Gemini:

```
Review this code for: bugs, security issues, performance problems,  
and code quality. Here are the key files:
```

```
[paste or reference the code]
```

Loop until reviewers are satisfied.

Step 6: Tests

Make sure tests exist and pass:

```
Ensure we have tests for all key functionality. Run them and fix any failures.
```

Claude will write tests if they don't exist and run them.

Step 7: Manual Verification

Ask Claude for manual testing instructions:

```
Give me step-by-step manual testing instructions so I can verify  
everything works correctly in the browser.
```

Then actually do the testing yourself. AI can't click buttons for you (yet).

Step 8: Commit

Once everything passes:

```
/commit
```

Or be specific:

```
Commit this with a clear message describing the expense tracker initial  
implementation.
```

Working with CLAUDE.md

`CLAUDE.md` is a special file that Claude Code reads automatically when it starts. Think of it as a briefing document — it tells Claude about your project before you say anything.

When to Create It

Create it early, ideally right after your first successful plan. You can ask Claude to create it:

Create a CLAUDE.md file for this project that captures our tech stack, coding conventions, and any rules we've established.

What to Put In It

```
# My Project

## Description
A personal expense tracker built with React and Vite.

## Tech Stack
- React 19 with Vite
- Tailwind CSS for styling
- Recharts for data visualization
- Local storage for persistence

## Project Structure
src/
  components/    - React components
  hooks/         - Custom hooks
  utils/         - Helper functions
  types/         - TypeScript types

## Conventions
- Use functional components with hooks
- Use TypeScript strict mode
- Tailwind for all styling (no CSS files)
- Named exports (not default exports)

## Rules
- Use latest stable versions of all dependencies
- Every component must have associated tests
- Keep components under 150 lines – split if larger
- Use descriptive variable names, no abbreviations
```

Why It Matters

Every time you start a new Claude Code session in that directory, Claude reads CLAUDE.md first. This means: - It doesn't "forget" your conventions between sessions - You get consistent code style across sessions - New rules you establish carry forward - It's also useful context for other AI tools reviewing your code

The Multi-AI Review Process

Using multiple AI models for review isn't overkill — it's how you catch more issues.

Why Multiple AIs?

- Each model has different training data and biases
- One might catch a security issue another misses
- Different models have different strengths (Claude: reasoning and planning, GPT: breadth of knowledge, Gemini: different perspective)
- It simulates having multiple human reviewers

How to Do It Efficiently

For plan reviews: Copy the plan text and paste it into each tool. Ask each to critique it.

For code reviews: Share the key files or a summary of changes. Ask each to focus on different aspects: - One reviewer: focus on bugs and correctness - Another reviewer: focus on architecture and maintainability - Both: flag security concerns

When reviewers disagree: This is valuable — it means there's a genuine tradeoff. Understand both perspectives and make your own decision. You're the architect.

Git & GitHub Practices

Core Principles

1. **Commit early and often** — Small commits are easier to understand and revert
2. **Clear commit messages** — Describe *what* and *why*, not *how*
3. **Create the GitHub repo early** — Even for personal projects, it's your backup

Setting Up a Repo

```
# Inside your project directory (after initial implementation)
git init
gh repo create my-project --public --source=. --push
```

Commit Rhythm

Good times to commit: - After the project scaffolding is set up - After each feature or component works - After code review fixes are applied - After tests are written and passing - After documentation updates

Branch Strategy (for bigger projects)

```
# Main branch stays stable
# Create feature branches for new work
git checkout -b feature/user-auth

# After feature is complete, reviewed, and tested:
# Create a PR via GitHub CLI
gh pr create --title "Add user authentication" --body "Description here"
```

Scaling to Bigger Projects

For anything beyond a single feature, break the work into phases.

Create a Roadmap

At the start of a bigger project, ask Claude:

Create a ROADMAP.md that breaks this project into development phases. Each phase should be independently buildable and testable. Order phases so each builds on the previous one.

Example ROADMAP.md:

```
# Project Roadmap

## Phase 1: Foundation (v0.1)
- [ ] Project setup and configuration
- [ ] Core data models
- [ ] Basic CRUD operations
- [x] Completed: 2026-03-04

## Phase 2: User Interface (v0.2)
- [ ] Main dashboard layout
- [ ] Data entry forms
- [ ] List/table views
- Status: In Progress

## Phase 3: Advanced Features (v0.3)
- [ ] Filtering and search
- [ ] Charts and visualizations
- [ ] Export functionality
- Status: Planned

## Phase 4: Polish (v1.0)
- [ ] Error handling and edge cases
- [ ] Performance optimization
- [ ] Final documentation
- Status: Planned
```

Update After Each Phase

After completing a phase:

Update ROADMAP.md – mark Phase 2 as complete with today's date, and update Phase 3 status to In Progress. Note any scope changes.

This keeps the AI (and you) oriented on what's done and what's next.

Each Phase Follows the Core Workflow

Every phase goes through the full cycle: plan → review → build → simplify → review → test → commit.

Documentation as a Superpower

Documentation isn't just for humans — it's critical for AI assistants.

Why Docs Matter More with AI

- **CLAUDE.md** tells Claude how to behave in your project
- **README.md** helps any AI understand what your project does
- **ROADMAP.md** keeps phased development on track
- **Inline comments** on tricky logic help AI understand intent (not just code)

- **API docs** let AI write correct integrations

What to Document

Document	When to Create	What Goes In It
CLAUDE.md	After first plan	Tech stack, conventions, rules
README.md	After first working version	What it is, how to run it, how to use it
ROADMAP.md	Start of multi-phase projects	Phases, status, scope changes
CHANGELOG.md	Optional, for bigger projects	What changed in each version

Keep Docs Updated

Stale docs are worse than no docs — they mislead the AI. After each significant change:

Update the README and CLAUDE.md to reflect the changes we just made.

Dealing with AI Limitations

Knowledge Cutoff

All AI models have a training data cutoff. They don't know about: - Libraries released or updated after their cutoff - New language features or framework changes - Recent security vulnerabilities

Workarounds: - Always specify: “use the latest stable version of [dependency]” - Use Context7 in Claude Code to fetch live documentation - If something doesn't compile/run, the API may have changed — check official docs - Paste relevant docs into the chat when needed

Hallucinations

Sometimes AI invents APIs, functions, or features that don't exist.

How to spot them: - Code that looks plausible but uses function names you can't find in docs - Import statements for packages that don't exist - Configuration options that aren't real

What to do: - If something looks suspicious, ask: “Does this API/function actually exist? Check the docs.” - Run the code — errors will surface hallucinated code quickly - Cross-reference with another AI model

Context Limits

Long conversations can cause the AI to “forget” earlier context.

Workarounds: - Keep CLAUDE.md comprehensive — it's always loaded - For long sessions, periodically summarize what you've accomplished - Start a fresh session for new features (Claude Code reads CLAUDE.md on start)

Prompt Patterns That Work

The Project Kickoff

```
I want to build [description].

Features:
- [feature 1]
- [feature 2]
- [feature 3]

Tech stack: [specifics]
Use latest stable versions of all dependencies.

Please enter plan mode and create an implementation plan.
```

The Feature Addition

```
I want to add [feature] to the existing project.
It should [behavior description].
Please enter plan mode first.
```

The Bug Fix

```
There's a bug: [describe what happens vs what should happen].
[Steps to reproduce if you have them]
Please investigate and fix it.
```

The Refactor Request

```
The [component/module] is getting too complex.
Please refactor it to be more maintainable.
Enter plan mode first – I want to approve the approach.
```

Specifying Versions

```
Use the latest stable version of Next.js, not an older one.
Check Context7 if you're unsure about current API patterns.
```

Common Mistakes to Avoid

1. **Skipping plan mode** — Going straight to code leads to rework. Always plan first.
2. **Only using one AI** — Cross-review with Codex and Gemini catches more issues.
3. **Giant prompts for giant features** — Break big things into phases. One feature at a time.
4. **Not committing often enough** — If something breaks, you want a recent good state to return to.
5. **Ignoring the simplifier** — `/simplify` catches things you won't notice. Run it.

6. **Trusting AI output blindly** — Always do manual testing. AI can write code that looks correct but isn't.
 7. **Forgetting to update docs** — Stale CLAUDE.md = Claude making wrong assumptions next session.
 8. **Not specifying dependency versions** — Say “latest stable” or you'll get whatever the model was trained on.
 9. **Trying to do everything in one session** — For big projects, work in phases. Start fresh sessions for new phases.
 10. **Not reading error messages** — When something fails, read the error. Often the fix is obvious and you can tell Claude exactly what went wrong.
-

Quick Start Checklist

For each new project:

- ☐ Create project directory
 - ☐ Start Claude with `claude --dangerously-skip-permissions`
 - ☐ Write a clear project description
 - ☐ Ask for plan mode
 - ☐ Review plan with Codex and Gemini
 - ☐ Approve and implement
 - ☐ Run `/simplify` until clean
 - ☐ Code review with Codex and Gemini
 - ☐ Ensure tests exist and pass
 - ☐ Manual testing
 - ☐ `git init` and commit
 - ☐ `gh repo create` and push
 - ☐ Create/update CLAUDE.md, README.md
 - ☐ For bigger projects: create ROADMAP.md
-